

Air Hockey Lab — LLM Chat Transcript

You

Here's my Air Hockey code (pasted). Task: fix puck–paddle collision in collisions.py. Currently it only flips vx, so the puck sticks/vibrates inside the paddle and ignores the hit angle. Use the collision normal (puck center – paddle center) to reflect velocity, only reflect when the puck is moving toward the paddle, and push the puck out to exactly puck.radius + paddle.radius. Also prevent tunneling at high speed. Change only collisions.py and, if needed, GameEngine.update.

Here's the fix. No tests were run, since your paste didn't include a runnable project here.

game/collisions.py

```
"""
collisions: puck-vs-paddle collision handling.
"""

import math

def handle_paddle_collision(puck, paddle):
    """
    If the puck overlaps the paddle, push it out to exactly
    puck.radius + paddle.radius and reflect its velocity about the
    collision normal (only if it's moving toward the paddle).
    Returns True if the circles overlapped this call.
    """
    min_dist = puck.radius + paddle.radius
    dx = puck.x - paddle.x
    dy = puck.y - paddle.y
    dist_sq = dx * dx + dy * dy

    if dist_sq >= min_dist * min_dist:
        return False

    dist = math.sqrt(dist_sq)
    if dist > 1e-9:
        nx, ny = dx / dist, dy / dist          # paddle -> puck
    else:
        # Centers coincide: fall back to "opposite of puck's motion".
        speed = math.hypot(puck.vx, puck.vy)
        if speed > 1e-9:
            nx, ny = -puck.vx / speed, -puck.vy / speed
        else:
            nx, ny = 1.0, 0.0

    # Resolve penetration: always, regardless of velocity direction.
    puck.x = paddle.x + nx * min_dist
    puck.y = paddle.y + ny * min_dist

    # Reflect only if approaching (v · n < 0). Prevents the sticking/
    # vibration where the velocity flips every frame while overlapping.
    vn = puck.vx * nx + puck.vy * ny
    if vn < 0:
        puck.vx -= 2 * vn * nx
        puck.vy -= 2 * vn * ny

    return True
```

GameEngine.update (add import math at the top of game_engine.py):

```
def update(self):
    self.ai.update(self.computer, self.puck)

    # Substep so the puck never moves more than half its radius per
    # step -> no tunneling through paddles or walls at high speed.
    speed = math.hypot(self.puck.vx, self.puck.vy)
```

```

steps = max(1, math.ceil(speed / (self.puck.radius * 0.5)))

for _ in range(steps):
    self.puck.x += self.puck.vx / steps
    self.puck.y += self.puck.vy / steps
    self.puck.bounce_off_walls(HEIGHT, MARGIN)

    handle_paddle_collision(self.puck, self.player)
    handle_paddle_collision(self.puck, self.computer)

self._handle_goals()

```

Why it works

Sticking/vibration: the old code flipped vx on every overlapping frame, so a puck that stayed inside the paddle flipped back and forth. Now it's pushed to exactly puck.radius + paddle.radius and only reflected when $v \cdot n < 0$.

Hit angle: $v' = v - 2(v \cdot n)n$ reflects about the real contact normal, so off-center hits deflect up or down.

Tunneling: the substep count scales with puck speed. Each step is at most 6 px (half the puck radius), far smaller than the 40 px minimum center distance. Walls and goal edges get the same protection.

Things to know

Substepped goals: I call `_handle_goals()` inside the loop so a fast puck can't skip past a wall. `_reset_puck` zeroes velocity, so the remaining substeps won't double-trigger. When you add scoring in Task 2, the reset has to keep zeroing or relaunching the velocity, or you'll double-count.

No paddle momentum: the paddle doesn't track its own velocity, so a hit only reflects the puck's speed and adds none. Smashes won't feel powerful. If you want that, store paddle vx/vy in `move_by` and add the relative-velocity term. That means touching `paddle.py`, so I left it out.

Wall pinning: a paddle can push the puck into a wall, and the wall bounce flips vy in the next substep. That's fine, but if you see jitter in corners, that's where it comes from.

You

Add scoring in `game_engine.py`. When the puck enters the left goal the computer scores; the right goal means the player scores. Keep `player_score` and `computer_score`, and draw both on screen with `renderer.draw_text`. Don't break the existing wall bounce outside the goal gap.

Adding scoring functionality to the game engine code.

Here are the changes, all in `game_engine.py`.

1. Score state in `__init__` (add after `self.ai = ComputerAI()`):

```

self.player_score = 0
self.computer_score = 0

```

2. Replace `_handle_goals` (the wall-bounce branches are unchanged; only the goal branches now award points):

```

def _handle_goals(self):
    # Left goal: puck went in on the player's side -> computer scores.
    if self.puck.x - self.puck.radius < MARGIN:
        if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
            self.computer_score += 1
            self._reset_puck()
        else:
            self.puck.x = MARGIN + self.puck.radius
            self.puck.vx = -self.puck.vx
    # Right goal: puck went in on the computer's side -> player scores.
    elif self.puck.x + self.puck.radius > WIDTH - MARGIN:
        if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
            self.player_score += 1

```

```

        self._reset_puck()
    else:
        self.puck.x = WIDTH - MARGIN - self.puck.radius
        self.puck.vx = -self.puck.vx

```

3. Replace draw:

```

def draw(self, surface, font):
    from game import renderer
    renderer.draw_table(surface)
    renderer.draw_paddle(surface, self.player, renderer.COLOR_PLAYER)
    renderer.draw_paddle(surface, self.computer, renderer.COLOR_COMPUTER)
    renderer.draw_puck(surface, self.puck)

    # Scores sit just inside the table, either side of the center line.
    y = MARGIN + 8
    renderer.draw_text(surface, font, str(self.player_score),
                       (WIDTH / 2 - 40, y), renderer.COLOR_PLAYER)
    renderer.draw_text(surface, font, str(self.computer_score),
                       (WIDTH / 2 + 22, y), renderer.COLOR_COMPUTER)

```

Notes

No double counting: update substeps the puck, and `_handle_goals` runs every substep. `_reset_puck` moves the puck to center and zeroes its velocity, so later substeps in the same frame can't re-trigger the goal. If you later make the reset relaunch the puck right away, the puck still starts at center, so it's still safe.

Wall bounce is untouched: a puck hitting the end wall outside the goal gap still gets clamped and has `vx` flipped, same as before.

The puck is dead after a goal: `_reset_puck` leaves `vx = vy = 0`, so after the first goal the puck just sits at center forever. That's the unfinished reset your starter docstring mentions, and I left it alone since you only asked for scoring. When you get to it, call `self._launch_puck()` at the end of `_reset_puck` (ideally after a short pause).

Goal detection uses the puck's center `y`, not its edges. A puck whose center is just outside the gap bounces even if part of it overlaps the opening. That's existing behavior, and I kept it.

Everything is in `game_engine.py`, and `main.py` stays untouched. Not run here, so give it a quick play.

1. Constants and imports (top of the file; math was added earlier, so only pygame and the duration are new):

```

import math
import random

import pygame

MATCH_DURATION_MS = 30_000

```

2. Replace `__init__` and add `reset()`, so R can reuse the same setup:

```

def __init__(self):
    self.reset()

def reset(self):
    """Start a fresh match: scores, timer, puck and paddles."""
    self.puck = Puck(WIDTH / 2, HEIGHT / 2, PUCK_RADIUS)
    self._launch_puck()

    self.player = Paddle(
        x=WIDTH * 0.15, y=HEIGHT / 2, radius=PADDLE_RADIUS,
        min_x=MARGIN + PADDLE_RADIUS, max_x=WIDTH / 2 - PADDLE_RADIUS,
        min_y=MARGIN + PADDLE_RADIUS, max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
    )
    self.computer = Paddle(
        x=WIDTH * 0.85, y=HEIGHT / 2, radius=PADDLE_RADIUS,
        min_x=WIDTH / 2 + PADDLE_RADIUS, max_x=WIDTH - MARGIN - PADDLE_RADIUS,
        min_y=MARGIN + PADDLE_RADIUS, max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
    )

```

```

self.ai = ComputerAI()

self.player_score = 0
self.computer_score = 0
self.game_over = False
self.start_ticks = pygame.time.get_ticks()

```

3. Timer helper (new method):

```

def _remaining_ms(self):
    if self.game_over:
        return 0
    elapsed = pygame.time.get_ticks() - self.start_ticks
    return max(0, MATCH_DURATION_MS - elapsed)

```

4. Replace handle_input (the R check comes first so it works after game over; movement is ignored once the match ends):

```

def handle_input(self, keys_pressed):
    if keys_pressed[pygame.K_r]:
        self.reset()
        return
    if self.game_over:
        return

    dx = dy = 0
    if keys_pressed[pygame.K_UP]:
        dy -= PLAYER_SPEED
    if keys_pressed[pygame.K_DOWN]:
        dy += PLAYER_SPEED
    if keys_pressed[pygame.K_LEFT]:
        dx -= PLAYER_SPEED
    if keys_pressed[pygame.K_RIGHT]:
        dx += PLAYER_SPEED
    self.player.move_by(dx, dy)

```

5. Top of update (the rest of the method stays as it was):

```

def update(self):
    if self.game_over:
        return
    if self._remaining_ms() <= 0:
        self.game_over = True
        return

    self.ai.update(self.computer, self.puck)
    # ... existing substep loop unchanged ...

```

6. Replace draw:

```

def draw(self, surface, font):
    from game import renderer
    renderer.draw_table(surface)
    renderer.draw_paddle(surface, self.player, renderer.COLOR_PLAYER)
    renderer.draw_paddle(surface, self.computer, renderer.COLOR_COMPUTER)
    renderer.draw_puck(surface, self.puck)

    y = MARGIN + 8
    renderer.draw_text(surface, font, str(self.player_score),
                      (WIDTH / 2 - 40, y), renderer.COLOR_PLAYER)
    renderer.draw_text(surface, font, str(self.computer_score),
                      (WIDTH / 2 + 22, y), renderer.COLOR_COMPUTER)

    # Remaining time, centered along the bottom of the table.
    seconds = math.ceil(self._remaining_ms() / 1000)
    label = f"Time: {seconds}"
    w, h = font.size(label)
    renderer.draw_text(surface, font, label,
                      (WIDTH / 2 - w / 2, HEIGHT - MARGIN - h - 8))

    if self.game_over:
        if self.player_score > self.computer_score:
            msg = "You Win"

```

```

elif self.computer_score > self.player_score:
    msg = "Computer Wins"
else:
    msg = "Draw"
renderer.draw_banner(surface, font, msg)
hint = "Press R to restart"
hw, _ = font.size(hint)
renderer.draw_text(surface, font, hint,
                   (WIDTH / 2 - hw / 2, HEIGHT / 2 + 30))

```

Notes

"Exactly 30s": the clock is wall-time from `get_ticks()`, so it doesn't drift with frame rate. The game-over check runs once per frame, so the match can end up to ~16 ms late at 60 FPS. That's the limit of any per-frame check.

Countdown display: it uses `ceil`, so you see $30 \rightarrow 1$, and the banner appears the instant it would show 0.

Frozen state: `update()` returns early, so the puck and AI stop, and `handle_input` ignores the arrows, so your paddle stops too.

R is polled, not event-driven: that keeps `main.py` untouched, but holding R resets the match every frame. A `KEYDOWN` handler in `main.py` that calls `engine.reset()` is the cleaner version if you're willing to touch that file.

Still open: `_reset_puck` leaves the puck dead after a goal, so after the first score the rest of the 30 seconds is a stationary puck. Add `self._launch_puck()` at the end of `_reset_puck` before you demo this, or the timer is just counting down on nothing.

You

`_reset_puck` leaves the puck motionless. After a goal, put the puck at the center and relaunch it using `_launch_puck()`, so velocity and direction are fully reset. Ideally serve toward the player who just conceded.

Resetting and relaunching the puck toward the conceding player.

Three small edits, all in `game_engine.py`. Not run here.

1. `_launch_puck` takes an optional direction (-1 = toward the player on the left, +1 = toward the computer; None keeps the old random behavior, so `reset()` still works):

```

def _launch_puck(self, direction=None):
    angle_choices = [0.3, 0.6, -0.3, -0.6]
    if direction is None:
        direction = random.choice([-1, 1])
    vy_factor = random.choice(angle_choices)
    self.puck.vx = INITIAL_PUCK_SPEED * direction
    self.puck.vy = INITIAL_PUCK_SPEED * vy_factor

```

2. Replace `_reset_puck`:

```

def _reset_puck(self, toward):
    """Re-center the puck and serve it toward the side that conceded."""
    self.puck.x, self.puck.y = WIDTH / 2, HEIGHT / 2
    self._launch_puck(toward)

```

3. Replace `_handle_goals` (the goal branches pass the serve direction and the method now returns whether a goal happened; the wall-bounce branches are unchanged):

```

def _handle_goals(self):
    """Returns True if a goal was scored this call."""
    # Left goal: player conceded -> computer scores, serve toward player.
    if self.puck.x - self.puck.radius < MARGIN:
        if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
            self.computer_score += 1
            self._reset_puck(toward=-1)

```

```

        return True
    self.puck.x = MARGIN + self.puck.radius
    self.puck.vx = -self.puck.vx
    # Right goal: computer conceded -> player scores, serve toward computer.
    elif self.puck.x + self.puck.radius > WIDTH - MARGIN:
        if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
            self.player_score += 1
            self._reset_puck(toward=1)
            return True
        self.puck.x = WIDTH - MARGIN - self.puck.radius
        self.puck.vx = -self.puck.vx
    return False

```

4. In update, stop substepping after a goal. Change the last line of the substep loop from `self._handle_goals()` to:

```

    if self._handle_goals():
        break

```

Why #4 matters: before, a reset zeroed the velocity, so leftover substeps in that frame did nothing. Now the puck has a fresh velocity, so without the break the remaining substeps would push it away from center before you ever see it. The substep count was also computed from the old, pre-goal speed.

Notes

Velocity is fully reset: `_launch_puck` overwrites both `vx` and `vy`, so nothing carries over from the shot that scored. Speed is back to `INITIAL_PUCK_SPEED` however hard the goal was.

Serve direction: if the computer scores in the left goal, the puck heads left at the player. If you score in the right goal, it heads right at the computer. The vertical angle is still random.

No pause before the serve: the puck launches the instant the goal registers, which can feel abrupt. A short delay (say, 1 s via `get_ticks()`, with the puck frozen at center) would fix it, but it touches update and the timer logic, so I left it out.

Paddle on the center line: a paddle can sit right at the center line, close enough to overlap the spawn point. The collision code pushes the puck out and reflects it, so it won't stick. It just means a camped paddle gets an instant touch on the serve.